gemstone-js vs gemstone-py

gemstone-js vs gemstone-py

`gemstone-js` and `gemstone-py` now overlap heavily for core GemStone/S work, but they are aimed at different application runtimes.

Use `gemstone-py` when the application is Python-first and you want the most mature product surface today: sync and async Python APIs, broader examples, FastAPI/Litestar/Django coverage, the Python database explorer, VS Code workbench flow, and the deeper PyPI/TestPyPI/native wheel release lane.

Use `gemstone-js` when the application is TypeScript or JavaScript-first: Node services, npm-distributed tools, explicit async request lifecycles, typed generated wrappers, web adapters, and package verification in a JS toolchain.

For object mapping, the projects should intentionally differ. `gemstone-py` can lean on Python's dynamic proxy style, while `gemstone-js` should prefer a connector-inspired mapping manifest that generates explicit async `*Ref` classes, repository helpers, and snapshot/dictionary helpers around `TypedOop<T>`. The lightweight `mappedObject()` helper is the current transparent layer: property-style async methods without synchronous property dispatch.

Quick Commands

```
gemstone-js-compare gemstone-js --scorecard
gemstone-js-compare --target gemstone-js --view scorecard
gemstone-js-compare gemstone-js --gaps
gemstone-js-compare gemstone-js --batches
gemstone-js-compare --target gemstone-js --scope beta --view totals
gemstone-js-compare gemstone-js --scorecard --markdown
gemstone-js-compare gemstone-js --totals --json
gemstone-js-compare --target gemstone-js --view totals --json --assert-total-batches 6 --assert-hours-max
gemstone-js-compare --target gemstone-js --scope beta --view totals --json --assert-total-batches 1 --asse
gemstone-js-compare --target gemstone-js --view totals --max-total-batches 6 --max-hours-max 72 --quiet
gemstone-js-compare --target gemstone-js --scope beta --view totals --max-total-batches 1 --max-hours-max
gemstone-js-compare gemstone-js --scorecard --json --output comparison-report.json
gemstone-js-compare gemstone-js --scorecard --format markdown --output comparison-report.md
gemstone-js-compare all --totals
```

Every JSON report includes `$schema: "./schemas/comparison-report.schema.json"` and `schema_version: 1`. `--output <path>` writes either JSON or human-readable reports for release notes and CI artifacts. `--markdown` or `--format markdown` renders Markdown directly. `npm run compare:check` validates every comparison target/view pair, output-file writing, Markdown rendering, exact assertion flags, maximum-threshold flags, quiet CI checks, and the current full and beta batch totals.

Current State

`gemstone-js` is close to `gemstone-py` for core database work:

- sessions, explicit login/logout, and environment-based configuration
- raw OOP handling, retained typed handles, and export-set cleanup
- value conversion for strings, symbols, numbers, arrays, dictionaries, dates, class instances, and custom converters
- persistent roots, dictionaries, ordered collections, GStore, ObjectLog, and reduced-conflict wrappers

- query helpers, indexes, bounded pages, count/exists/first/limit helpers, and collection mutation helpers
- migrations, transaction retries, nested transactions, request scopes, pools, and web adapter lifecycles
- manifest/decorator TypeScript codegen, JSON Schemas, examples, benchmark reports, checksums, release artifact checks, and installed package probes
- bulk selector sends for raw OOPs, marshalled JS values, mixed calls, and object-returning retained handles
- a small dependency-free browser explorer for status, OOP inspection, roots/globals, workspace eval, class browsing, and codegen preview

`gemstone-py` remains ahead where project maturity matters:

- broader real application examples
- Python async and sync surfaces
- framework coverage, especially Django and larger Flask/webstack examples
- more mature visual explorer and VS Code workflows
- release history and native wheel confidence
- broader live GemStone test coverage across application shapes

Work Remaining

The comparison CLI reports **6 batches**, roughly **42-72 hours**, for fuller JS/Python product parity:

1. Native publish confidence
2. Visual tooling polish
3. Installed examples
4. Live CI
5. Documentation and release polish
6. Cross-project alignment

The most useful post-beta API polish batch is object mapping maturity: publish a mapping manifest schema, generate `*Ref` classes wrapping `TypedOop<T>`, add repository helpers returning typed refs, generate bounded snapshots for UI/API payloads, and surface the mapping catalog in Explorer or the VS Code workbench.

For a narrower conservative JavaScript beta, `--scope beta` now tracks one remaining validation batch, roughly **4-8 hours**: run the release candidate from clean installed JS and native artifacts, verify worker-mode live smoke, and inspect release metadata before publishing.

Practical Recommendation

For production Python projects, start with `gemstone-py`.

For TypeScript services, npm packages, Node tooling, or codegen-heavy JS workflows, use `gemstone-js`, but validate native behavior against a real Stone before treating it as production-stable.

The VS Code extension should remain a later track. The JavaScript API and native session-thread behavior should settle first so editor workflows do not freeze unstable assumptions.

For the implementation-level parity ledger, see `docs/gemstone-py-parity.md`. For the Rust/Python ecosystem comparison, see `docs/gemstone-rs-comparison.md`.